

15_comparacion_cualitativa externa_vs_interna

May 19, 2026

1 15. Evaluación cualitativa conciliada

En este notebook se comparan dos evaluaciones cualitativas aplicadas a las salidas generadas por los sistemas de prompting.

Se utilizan dos fuentes:

- Evaluación cualitativa interna.
- Evaluación cualitativa externa.

Ambas evaluaciones contienen 144 registros correspondientes a 36 textos evaluados en 4 configuraciones prompt-based.

```
[27]: from pathlib import Path
import pandas as pd
import numpy as np

PROJECT_ROOT = Path("/home/harielpadillasanchez/Documentos/TT/TT2")

CUALITATIVA_DIR = PROJECT_ROOT / "data" / "cualitativa"

FILE_INTERNA = CUALITATIVA_DIR / "Evaluacion_cualitativa_interna.xlsx"
FILE_EXTERNA = CUALITATIVA_DIR / "Evaluacion_cualitativa_externa.xlsx"

OUT_DIR = PROJECT_ROOT / "outputs" / "evaluacion_cualitativa_conciliada"
OUT_DIR.mkdir(parents=True, exist_ok=True)

print("PROJECT_ROOT:", PROJECT_ROOT)
print("FILE_INTERNA:", FILE_INTERNA, "| existe:", FILE_INTERNA.exists())
print("FILE_EXTERNA:", FILE_EXTERNA, "| existe:", FILE_EXTERNA.exists())
print("OUT_DIR:", OUT_DIR)
```

```
PROJECT_ROOT: /home/harielpadillasanchez/Documentos/TT/TT2
FILE_INTERNA: /home/harielpadillasanchez/Documentos/TT/TT2/data/cualitativa/Evaluacion_cualitativa_interna.xlsx | existe: True
FILE_EXTERNA: /home/harielpadillasanchez/Documentos/TT/TT2/data/cualitativa/Evaluacion_cualitativa_externa.xlsx | existe: True
OUT_DIR: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/evaluacion_cualitativa_conciliada
```

```
[28]: SHEET_NAME = "Evaluación cualitativa"

TEXT_COL = "Texto"

CRITERIA_COLS = [
    "Semántica (5|3|1)",
    "Claridad (5|3|1)",
    "Fluidez y corrección (5|3|1)",
    "Léxico y sintaxis (5|3|1)",
    "Adecuación al público / tono (5|3|1)",
    "Estructura y presentación (5|3|1)",
    "Manejo de números y datos (5|3|1)",
]

WEIGHTS = {
    "Semántica (5|3|1)": 0.25,
    "Claridad (5|3|1)": 0.25,
    "Fluidez y corrección (5|3|1)": 0.15,
    "Léxico y sintaxis (5|3|1)": 0.10,
    "Adecuación al público / tono (5|3|1)": 0.10,
    "Estructura y presentación (5|3|1)": 0.10,
    "Manejo de números y datos (5|3|1)": 0.05,
}

assert abs(sum(WEIGHTS.values()) - 1.0) < 1e-9

print("Hoja a leer:", SHEET_NAME)
print("Número de criterios:", len(CRITERIA_COLS))
print("Suma de pesos:", sum(WEIGHTS.values()))

pd.DataFrame({
    "criterio": CRITERIA_COLS,
    "peso": [WEIGHTS[c] for c in CRITERIA_COLS]
})
```

Hoja a leer: Evaluación cualitativa

Número de criterios: 7

Suma de pesos: 1.0

```
[28]:
```

	criterio	peso
0	Semántica (5 3 1)	0.25
1	Claridad (5 3 1)	0.25
2	Fluidez y corrección (5 3 1)	0.15
3	Léxico y sintaxis (5 3 1)	0.10
4	Adecuación al público / tono (5 3 1)	0.10
5	Estructura y presentación (5 3 1)	0.10
6	Manejo de números y datos (5 3 1)	0.05

```
[29]: def read_qualitative_eval(path, sheet_name=SHEET_NAME):
    """
    Lee una evaluación cualitativa en formato .xlsx, .xls o .csv.

    Para archivos Excel, lee únicamente la hoja 'Evaluación cualitativa'.

    La estructura esperada es:
    - Fila 1: encabezados.
    - Fila 2: pesos.
    - Filas posteriores: evaluaciones de los textos 1 a 144.

    La función elimina la fila de pesos y conserva solo los registros cuyo campo
    'Texto' sea numérico y esté entre 1 y 144.
    """

    path = Path(path)

    if not path.exists():
        raise FileNotFoundError(f"No existe el archivo: {path}")

    if path.suffix.lower() in [".xlsx", ".xls"]:
        df = pd.read_excel(path, sheet_name=sheet_name)
    elif path.suffix.lower() == ".csv":
        df = pd.read_csv(path)
    else:
        raise ValueError("Formato no soportado. Usa .xlsx, .xls o .csv")

    df = df.copy()

    # Eliminar columnas completamente vacías
    df = df.dropna(axis=1, how="all")

    # Validar columnas requeridas
    required_cols = [TEXT_COL] + CRITERIA_COLS
    missing = [col for col in required_cols if col not in df.columns]

    if missing:
        raise ValueError(
            "Faltan columnas requeridas:\n"
            + "\n".join(missing)
            + "\n\nColumnas encontradas:\n"
            + "\n".join(df.columns.astype(str))
        )

    # Eliminar fila de pesos y filas vacías.
    # Solo se conservan filas donde 'Texto' es numérico.
    df = df[pd.to_numeric(df[TEXT_COL], errors="coerce").notna()].copy()
```

```

# Convertir Texto a entero
df[TEXT_COL] = pd.to_numeric(df[TEXT_COL], errors="coerce").astype(int)

# Conservar solo los 144 registros de evaluación prompt-based
df = df[df[TEXT_COL].between(1, 144)].copy()

# Ordenar por número de texto
df = df.sort_values(TEXT_COL).reset_index(drop=True)

# Convertir criterios a valores numéricos
for col in CRITERIA_COLS:
    df[col] = pd.to_numeric(df[col], errors="coerce")

return df

```

```

[30]: df_interna = read_qualitative_eval(FILE_INTERNA)
df_externa = read_qualitative_eval(FILE_EXTERNA)

print("Shape evaluación interna:", df_interna.shape)
print("Shape evaluación externa:", df_externa.shape)

print("\nPrimeras filas evaluación interna:")
display(df_interna.head())

print("\nPrimeras filas evaluación externa:")
display(df_externa.head())

```

Shape evaluación interna: (144, 11)

Shape evaluación externa: (144, 10)

Primeras filas evaluación interna:

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
0	1	3.0	5.0	
1	2	3.0	5.0	
2	3	5.0	5.0	
3	4	3.0	5.0	
4	5	5.0	5.0	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
--	--------------------------------------	-----------------------------------	---

0	5.0	5.0
1	5.0	5.0
2	5.0	5.0
3	5.0	5.0
4	5.0	5.0

	Manejo de números y datos (5 3 1)	Puntaje Final	Unnamed: 9	Unnamed: 10
0	5.0	4.5	NaN	NaN
1	5.0	4.5	NaN	NaN
2	5.0	5.0	NaN	NaN
3	5.0	4.5	NaN	NaN
4	5.0	5.0	NaN	NaN

Primeras filas evaluación externa:

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
0	1	5.0	5.0	
1	2	5.0	5.0	
2	3	5.0	5.0	
3	4	5.0	3.0	
4	5	5.0	5.0	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	1.0	
4	5.0	5.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	

	Manejo de números y datos (5 3 1)	Puntaje Final	Unnamed: 10
0	5.0	5.0	NaN
1	5.0	5.0	NaN
2	5.0	5.0	NaN
3	5.0	4.1	NaN
4	5.0	5.0	NaN

```
[31]: print("Validación de tamaños:")

if len(df_interna) != 144:
```

```

    print(f"Advertencia: la evaluación interna tiene {len(df_interna)} filas,
↪no 144.")
else:
    print("OK: evaluación interna tiene 144 filas.")

if len(df_externa) != 144:
    print(f"Advertencia: la evaluación externa tiene {len(df_externa)} filas,
↪no 144.")
else:
    print("OK: evaluación externa tiene 144 filas.")

print("\nRango de textos:")
print("Interna:", df_interna[TEXT_COL].min(), "a", df_interna[TEXT_COL].max())
print("Externa:", df_externa[TEXT_COL].min(), "a", df_externa[TEXT_COL].max())

print("\nValores únicos por criterio en evaluación interna:")
for col in CRITERIA_COLS:
    print(col, sorted(df_interna[col].dropna().unique().tolist()))

print("\nValores únicos por criterio en evaluación externa:")
for col in CRITERIA_COLS:
    print(col, sorted(df_externa[col].dropna().unique().tolist()))

```

Validación de tamaños:

OK: evaluación interna tiene 144 filas.

OK: evaluación externa tiene 144 filas.

Rango de textos:

Interna: 1 a 144

Externa: 1 a 144

Valores únicos por criterio en evaluación interna:

Semántica (5|3|1) [0.0, 1.0, 3.0, 5.0]

Claridad (5|3|1) [0.0, 1.0, 3.0, 5.0]

Fluidez y corrección (5|3|1) [0.0, 1.0, 3.0, 5.0]

Léxico y sintaxis (5|3|1) [0.0, 3.0, 5.0]

Adecuación al público / tono (5|3|1) [0.0, 5.0]

Estructura y presentación (5|3|1) [0.0, 3.0, 5.0]

Manejo de números y datos (5|3|1) [0.0, 1.0, 3.0, 5.0]

Valores únicos por criterio en evaluación externa:

Semántica (5|3|1) [0.0, 1.0, 3.0, 5.0]

Claridad (5|3|1) [0.0, 1.0, 3.0, 5.0]

Fluidez y corrección (5|3|1) [0.0, 1.0, 3.0, 5.0]

Léxico y sintaxis (5|3|1) [0.0, 1.0, 3.0, 5.0]

Adecuación al público / tono (5|3|1) [0.0, 1.0, 3.0, 5.0]

Estructura y presentación (5|3|1) [0.0, 1.0, 3.0, 5.0]

Manejo de números y datos (5|3|1) [0.0, 1.0, 3.0, 5.0]

```
[32]: merged = df_interna[[TEXT_COL] + CRITERIA_COLS].merge(
    df_externa[[TEXT_COL] + CRITERIA_COLS],
    on=TEXT_COL,
    suffixes=("_interna", "_externa"),
    how="outer",
    indicator=True
)

print("Shape merged:", merged.shape)
print("\nCoincidencias:")
print(merged["_merge"].value_counts())

display(merged.head())
```

Shape merged: (144, 16)

Coincidencias:

_merge

both 144

left_only 0

right_only 0

Name: count, dtype: int64

	Texto	Semántica	(5 3 1)_interna	Claridad	(5 3 1)_interna \
0	1		3.0		5.0
1	2		3.0		5.0
2	3		5.0		5.0
3	4		3.0		5.0
4	5		5.0		5.0

	Fluidez y corrección	(5 3 1)_interna	Léxico y sintaxis	(5 3 1)_interna \
0		5.0		5.0
1		5.0		5.0
2		5.0		5.0
3		5.0		5.0
4		5.0		5.0

	Adecuación al público / tono	(5 3 1)_interna \
0		5.0
1		5.0
2		5.0
3		5.0
4		5.0

	Estructura y presentación	(5 3 1)_interna \
0		5.0
1		5.0

2	5.0
3	5.0
4	5.0

	Manejo de números y datos (5 3 1)_interna	Semántica (5 3 1)_externa \
0	5.0	5.0
1	5.0	5.0
2	5.0	5.0
3	5.0	5.0
4	5.0	5.0

	Claridad (5 3 1)_externa	Fluidez y corrección (5 3 1)_externa \
0	5.0	5.0
1	5.0	5.0
2	5.0	5.0
3	3.0	5.0
4	5.0	5.0

	Léxico y sintaxis (5 3 1)_externa \
0	5.0
1	5.0
2	5.0
3	1.0
4	5.0

	Adecuación al público / tono (5 3 1)_externa \
0	5.0
1	5.0
2	5.0
3	5.0
4	5.0

	Estructura y presentación (5 3 1)_externa \
0	5.0
1	5.0
2	5.0
3	5.0
4	5.0

	Manejo de números y datos (5 3 1)_externa _merge
0	5.0 both
1	5.0 both
2	5.0 both
3	5.0 both
4	5.0 both

```
[33]: if not (merged["_merge"] == "both").all():
        print("Advertencia: hay textos que no aparecen en ambas evaluaciones.")
        display(merged[merged["_merge"] != "both"])
    else:
        print("OK: todos los textos aparecen en ambas evaluaciones.")
```

OK: todos los textos aparecen en ambas evaluaciones.

```
[34]: def conciliacion_con_regla_cero(score_interna, score_externa):
        """
        Aplica la regla de conciliación:

        - Si cualquiera de las dos evaluaciones asigna 0, el valor final es 0.
        - Si ninguna asigna 0, el valor final es el promedio.
        - Si ambos valores están vacíos, el resultado queda como NaN.

        Retorna:
        - valor_final
        - aplico_regla_cero
        """

        if pd.isna(score_interna) and pd.isna(score_externa):
            return np.nan, False

        if score_interna == 0 or score_externa == 0:
            return 0, True

        values = [
            value
            for value in [score_interna, score_externa]
            if pd.notna(value)
        ]

        if len(values) == 0:
            return np.nan, False

        return float(np.mean(values)), False
```

```
[35]: rows = []

for _, row in merged.iterrows():
    result = {
        "Texto": row[TEXT_COL],
    }

    criterios_con_cero = []

    for criterion in CRITERIA_COLS:
```

```

score_interna = row[f"{criterion}_interna"]
score_externa = row[f"{criterion}_externa"]

final_score, applied_zero = conciliacion_con_regla_cero(
    score_interna,
    score_externa
)

result[f"{criterion}_interna"] = score_interna
result[f"{criterion}_externa"] = score_externa
result[criterion] = final_score

result[f"{criterion}_diferencia_original"] = (
    abs(score_interna - score_externa)
    if pd.notna(score_interna) and pd.notna(score_externa)
    else np.nan
)

result[f"{criterion}_aplico_regla_cero"] = applied_zero

criterios_con_cero.append(applied_zero)

# Puntaje final ponderado usando los criterios conciliados
puntaje_final = 0

for criterion in CRITERIA_COLS:
    value = result[criterion]

    if pd.notna(value):
        puntaje_final += value * WEIGHTS[criterion]

result["Puntaje Final"] = puntaje_final
result["Aplico regla cero"] = any(criterios_con_cero)

rows.append(result)

df_conciliada = pd.DataFrame(rows)
df_conciliada = df_conciliada.sort_values("Texto").reset_index(drop=True)

print("Shape evaluación conciliada:", df_conciliada.shape)
display(df_conciliada.head())

```

Shape evaluación conciliada: (144, 38)

	Texto	Semántica	(5 3 1)_interna	Semántica	(5 3 1)_externa	\
0	1		3.0		5.0	
1	2		3.0		5.0	
2	3		5.0		5.0	

3	4	3.0	5.0
4	5	5.0	5.0

	Semántica (5 3 1)	Semántica (5 3 1)_diferencia_original \
0	4.0	2.0
1	4.0	2.0
2	5.0	0.0
3	4.0	2.0
4	5.0	0.0

	Semántica (5 3 1)_aplico_regla_cero	Claridad (5 3 1)_interna \
0	False	5.0
1	False	5.0
2	False	5.0
3	False	5.0
4	False	5.0

	Claridad (5 3 1)_externa	Claridad (5 3 1) \
0	5.0	5.0
1	5.0	5.0
2	5.0	5.0
3	3.0	4.0
4	5.0	5.0

	Claridad (5 3 1)_diferencia_original ... \
0	0.0 ...
1	0.0 ...
2	0.0 ...
3	2.0 ...
4	0.0 ...

	Estructura y presentación (5 3 1) \
0	5.0
1	5.0
2	5.0
3	5.0
4	5.0

	Estructura y presentación (5 3 1)_diferencia_original \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0

	Estructura y presentación (5 3 1)_aplico_regla_cero \
0	False
1	False

2	False
3	False
4	False

Manejo de números y datos (5 3 1)_interna \	
0	5.0
1	5.0
2	5.0
3	5.0
4	5.0

Manejo de números y datos (5 3 1)_externa \	
0	5.0
1	5.0
2	5.0
3	5.0
4	5.0

Manejo de números y datos (5 3 1) \	
0	5.0
1	5.0
2	5.0
3	5.0
4	5.0

Manejo de números y datos (5 3 1)_diferencia_original \	
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0

Manejo de números y datos (5 3 1)_aplico_regla_cero Puntaje Final \		
0	False	4.75
1	False	4.75
2	False	5.00
3	False	4.30
4	False	5.00

Aplico regla cero	
0	False
1	False
2	False
3	False
4	False

[5 rows x 38 columns]

```
[36]: final_cols = ["Texto"] + CRITERIA_COLS + ["Puntaje Final", "Aplico regla cero"]

df_eval_final = df_conciliada[final_cols].copy()

print("Shape tabla final:", df_eval_final.shape)
display(df_eval_final.head())
```

Shape tabla final: (144, 10)

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
0	1	4.0	5.0	
1	2	4.0	5.0	
2	3	5.0	5.0	
3	4	4.0	4.0	
4	5	5.0	5.0	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	3.0	
4	5.0	5.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	

	Manejo de números y datos (5 3 1)	Puntaje Final	Aplico regla cero
0	5.0	4.75	False
1	5.0	4.75	False
2	5.0	5.00	False
3	5.0	4.30	False
4	5.0	5.00	False

```
[37]: print("Resumen estadístico de criterios conciliados:")
display(df_eval_final[CRITERIA_COLS + ["Puntaje Final"]].describe())

print("Número de textos donde se aplicó al menos una regla de cero:")
print(df_eval_final["Aplico regla cero"].sum())
```

Resumen estadístico de criterios conciliados:

	Semántica (5 3 1)	Claridad (5 3 1)	Fluidez y corrección (5 3 1)	\
count	144.000000	144.000000	144.000000	
mean	3.236111	3.423611	3.638889	
std	1.992946	1.984485	1.973996	

min	0.000000	0.000000	0.000000
25%	1.000000	2.000000	3.000000
50%	4.000000	4.000000	5.000000
75%	5.000000	5.000000	5.000000
max	5.000000	5.000000	5.000000

	Léxico y sintaxis (5 3 1)	Adecuación al público / tono (5 3 1)	\
count	144.000000	144.000000	
mean	3.750000	3.833333	
std	1.977142	1.989483	
min	0.000000	0.000000	
25%	3.000000	3.750000	
50%	5.000000	5.000000	
75%	5.000000	5.000000	
max	5.000000	5.000000	

	Estructura y presentación (5 3 1)	Manejo de números y datos (5 3 1)	\
count	144.000000	144.000000	
mean	3.770833	3.673611	
std	1.981448	2.023740	
min	0.000000	0.000000	
25%	3.000000	3.000000	
50%	5.000000	5.000000	
75%	5.000000	5.000000	
max	5.000000	5.000000	

	Puntaje Final
count	144.000000
mean	3.529861
std	1.909805
min	0.000000
25%	2.637500
50%	4.500000
75%	4.950000
max	5.000000

Número de textos donde se aplicó al menos una regla de cero:

29

```
[38]: diff_rows = []

for criterion in CRITERIA_COLS:
    diff_col = f"{criterion}_diferencia_original"
    zero_col = f"{criterion}_aplico_regla_cero"

    diff_rows.append({
        "criterio": criterion,
        "diferencia_promedio": df_conciliada[diff_col].mean(),
```

```

        "diferencia_mediana": df_conciliada[diff_col].median(),
        "diferencia_maxima": df_conciliada[diff_col].max(),
        "casos_con_diferencia": int((df_conciliada[diff_col].fillna(0) > 0).
→sum()),
        "casos_sin_diferencia": int((df_conciliada[diff_col].fillna(0) == 0).
→sum()),
        "casos_con_regla_cero": int(df_conciliada[zero_col].sum()),
    })

df_resumen_diferencias = pd.DataFrame(diff_rows)

display(df_resumen_diferencias)

```

	criterio	diferencia_promedio \
0	Semántica (5 3 1)	1.229167
1	Claridad (5 3 1)	0.993056
2	Fluidez y corrección (5 3 1)	1.076389
3	Léxico y sintaxis (5 3 1)	1.062500
4	Adecuación al público / tono (5 3 1)	1.048611
5	Estructura y presentación (5 3 1)	1.076389
6	Manejo de números y datos (5 3 1)	1.119718

	diferencia_mediana	diferencia_maxima	casos_con_diferencia \
0	0.0	5.0	58
1	0.0	5.0	51
2	0.0	5.0	49
3	0.0	5.0	40
4	0.0	5.0	37
5	0.0	5.0	40
6	0.0	5.0	40

	casos_sin_diferencia	casos_con_regla_cero
0	86	29
1	93	29
2	95	29
3	104	29
4	107	29
5	104	29
6	104	29

```

[39]: diff_cols = [
        f"{criterion}_diferencia_original"
        for criterion in CRITERIA_COLS
    ]

zero_cols = [
    f"{criterion}_aplico_regla_cero"

```

```

    for criterion in CRITERIA_COLS
]

all_differences = df_conciliada[diff_cols].stack()

df_resumen_general = pd.DataFrame([
    {
        "n_textos": len(df_eval_final),
        "n_criterios": len(CRITERIA_COLS),
        "n_comparaciones": len(df_eval_final) * len(CRITERIA_COLS),
        "diferencia_promedio_global": all_differences.mean(),
        "diferencia_mediana_global": all_differences.median(),
        "diferencia_maxima_global": all_differences.max(),
        "total_casos_con_diferencia": int((all_differences > 0).sum()),
        "total_casos_sin_diferencia": int((all_differences == 0).sum()),
        "total_casos_con_regla_cero": int(df_conciliada[zero_cols].sum().sum()),
        "textos_con_alguna_regla_cero": int(df_conciliada["Aplico regla cero"].
↪sum()),
        "puntaje_promedio_final": df_eval_final["Puntaje Final"].mean(),
        "puntaje_mediano_final": df_eval_final["Puntaje Final"].median(),
        "puntaje_minimo_final": df_eval_final["Puntaje Final"].min(),
        "puntaje_maximo_final": df_eval_final["Puntaje Final"].max(),
    }
])

display(df_resumen_general)

```

```

n_textos  n_criterios  n_comparaciones  diferencia_promedio_global  \
0         144          7             1008              1.086481

diferencia_mediana_global  diferencia_maxima_global  \
0                        0.0                      5.0

total_casos_con_diferencia  total_casos_sin_diferencia  \
0                        315                      691

total_casos_con_regla_cero  textos_con_alguna_regla_cero  \
0                        203                      29

puntaje_promedio_final  puntaje_mediano_final  puntaje_minimo_final  \
0              3.529861                4.5              0.0

puntaje_maximo_final
0                5.0

```

```
[40]: casos_diferencia = []
```

```

for _, row in df_conciliada.iterrows():
    for criterion in CRITERIA_COLS:
        diff = row[f"{criterion}_diferencia_original"]

        if pd.notna(diff) and diff > 0:
            casos_diferencia.append({
                "Texto": row["Texto"],
                "criterio": criterion,
                "evaluacion_interna": row[f"{criterion}_interna"],
                "evaluacion_externa": row[f"{criterion}_externa"],
                "diferencia": diff,
                "valor_final_conciliado": row[criterion],
                "aplico_regla_cero": row[f"{criterion}_aplico_regla_cero"],
            })

df_casos_diferencia = pd.DataFrame(casos_diferencia)

if len(df_casos_diferencia) > 0:
    df_casos_diferencia = df_casos_diferencia.sort_values(
        ["diferencia", "Texto", "criterio"],
        ascending=[False, True, True]
    ).reset_index(drop=True)

print("Número de diferencias encontradas:", len(df_casos_diferencia))
display(df_casos_diferencia.head(30))

```

Número de diferencias encontradas: 315

	Texto	criterio	evaluacion_interna \
0	72 Adecuación al público / tono	(5 3 1)	5.0
1	72 Claridad	(5 3 1)	5.0
2	72 Estructura y presentación	(5 3 1)	5.0
3	72 Fluidez y corrección	(5 3 1)	5.0
4	72 Léxico y sintaxis	(5 3 1)	5.0
5	72 Manejo de números y datos	(5 3 1)	5.0
6	72 Semántica	(5 3 1)	5.0
7	73 Adecuación al público / tono	(5 3 1)	0.0
8	73 Claridad	(5 3 1)	0.0
9	73 Estructura y presentación	(5 3 1)	0.0
10	73 Fluidez y corrección	(5 3 1)	0.0
11	73 Manejo de números y datos	(5 3 1)	0.0
12	73 Semántica	(5 3 1)	0.0
13	79 Adecuación al público / tono	(5 3 1)	5.0
14	79 Estructura y presentación	(5 3 1)	5.0
15	79 Léxico y sintaxis	(5 3 1)	5.0
16	79 Manejo de números y datos	(5 3 1)	5.0
17	84 Adecuación al público / tono	(5 3 1)	5.0
18	84 Claridad	(5 3 1)	5.0

19	84	Estructura y presentación (5 3 1)	5.0
20	84	Fluidez y corrección (5 3 1)	5.0
21	84	Léxico y sintaxis (5 3 1)	5.0
22	84	Manejo de números y datos (5 3 1)	5.0
23	84	Semántica (5 3 1)	5.0
24	89	Adecuación al público / tono (5 3 1)	5.0
25	89	Claridad (5 3 1)	5.0
26	89	Estructura y presentación (5 3 1)	5.0
27	89	Fluidez y corrección (5 3 1)	5.0
28	89	Léxico y sintaxis (5 3 1)	5.0
29	89	Manejo de números y datos (5 3 1)	5.0

	evaluacion_externa	diferencia	valor_final_conciliado	aplico_regla_cero
0	0.0	5.0	0.0	True
1	0.0	5.0	0.0	True
2	0.0	5.0	0.0	True
3	0.0	5.0	0.0	True
4	0.0	5.0	0.0	True
5	0.0	5.0	0.0	True
6	0.0	5.0	0.0	True
7	5.0	5.0	0.0	True
8	5.0	5.0	0.0	True
9	5.0	5.0	0.0	True
10	5.0	5.0	0.0	True
11	5.0	5.0	0.0	True
12	5.0	5.0	0.0	True
13	0.0	5.0	0.0	True
14	0.0	5.0	0.0	True
15	0.0	5.0	0.0	True
16	0.0	5.0	0.0	True
17	0.0	5.0	0.0	True
18	0.0	5.0	0.0	True
19	0.0	5.0	0.0	True
20	0.0	5.0	0.0	True
21	0.0	5.0	0.0	True
22	0.0	5.0	0.0	True
23	0.0	5.0	0.0	True
24	0.0	5.0	0.0	True
25	0.0	5.0	0.0	True
26	0.0	5.0	0.0	True
27	0.0	5.0	0.0	True
28	0.0	5.0	0.0	True
29	0.0	5.0	0.0	True

```
[41]: casos_regla_cero = []

for _, row in df_conciliada.iterrows():
```

```

for criterion in CRITERIA_COLS:
    applied_zero = row[f"{criterion}_aplico_regla_cero"]

    if applied_zero:
        casos_regla_cero.append({
            "Texto": row["Texto"],
            "criterio": criterion,
            "evaluacion_interna": row[f"{criterion}_interna"],
            "evaluacion_externa": row[f"{criterion}_externa"],
            "valor_final_conciliado": row[criterion],
        })

df_casos_regla_cero = pd.DataFrame(casos_regla_cero)

print("Número de casos donde se aplicó regla del 0:", len(df_casos_regla_cero))
display(df_casos_regla_cero.head(30))

```

Número de casos donde se aplicó regla del 0: 203

	Texto	criterio	evaluacion_interna \
0	72	Semántica (5 3 1)	5.0
1	72	Claridad (5 3 1)	5.0
2	72	Fluidez y corrección (5 3 1)	5.0
3	72	Léxico y sintaxis (5 3 1)	5.0
4	72	Adecuación al público / tono (5 3 1)	5.0
5	72	Estructura y presentación (5 3 1)	5.0
6	72	Manejo de números y datos (5 3 1)	5.0
7	73	Semántica (5 3 1)	0.0
8	73	Claridad (5 3 1)	0.0
9	73	Fluidez y corrección (5 3 1)	0.0
10	73	Léxico y sintaxis (5 3 1)	0.0
11	73	Adecuación al público / tono (5 3 1)	0.0
12	73	Estructura y presentación (5 3 1)	0.0
13	73	Manejo de números y datos (5 3 1)	0.0
14	79	Semántica (5 3 1)	3.0
15	79	Claridad (5 3 1)	3.0
16	79	Fluidez y corrección (5 3 1)	3.0
17	79	Léxico y sintaxis (5 3 1)	5.0
18	79	Adecuación al público / tono (5 3 1)	5.0
19	79	Estructura y presentación (5 3 1)	5.0
20	79	Manejo de números y datos (5 3 1)	5.0
21	81	Semántica (5 3 1)	0.0
22	81	Claridad (5 3 1)	0.0
23	81	Fluidez y corrección (5 3 1)	0.0
24	81	Léxico y sintaxis (5 3 1)	0.0
25	81	Adecuación al público / tono (5 3 1)	0.0
26	81	Estructura y presentación (5 3 1)	0.0
27	81	Manejo de números y datos (5 3 1)	0.0

28	83	Semántica (5 3 1)	0.0
29	83	Claridad (5 3 1)	0.0

	evaluacion_externa	valor_final_conciliado
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	0.0
6	0.0	0.0
7	5.0	0.0
8	5.0	0.0
9	5.0	0.0
10	3.0	0.0
11	5.0	0.0
12	5.0	0.0
13	5.0	0.0
14	0.0	0.0
15	0.0	0.0
16	0.0	0.0
17	0.0	0.0
18	0.0	0.0
19	0.0	0.0
20	0.0	0.0
21	0.0	0.0
22	0.0	0.0
23	0.0	0.0
24	0.0	0.0
25	0.0	0.0
26	0.0	0.0
27	0.0	0.0
28	0.0	0.0
29	0.0	0.0

```
[42]: SYSTEM_BLOCKS = [
    {
        "start": 1,
        "end": 36,
        "system_id": "prompt_1",
        "system_name": "PROMPTING_SISTEMA_1",
        "system_family": "prompting",
    },
    {
        "start": 37,
        "end": 72,
        "system_id": "prompt_2",
```

```

        "system_name": "PROMPTING_SISTEMA_2",
        "system_family": "prompting",
    },
    {
        "start": 73,
        "end": 108,
        "system_id": "prompt_3",
        "system_name": "PROMPTING_SISTEMA_3",
        "system_family": "prompting",
    },
    {
        "start": 109,
        "end": 144,
        "system_id": "prompt_4",
        "system_name": "PROMPTING_SISTEMA_4",
        "system_family": "prompting",
    },
]

def assign_system(text_number):
    for block in SYSTEM_BLOCKS:
        if block["start"] <= text_number <= block["end"]:
            return pd.Series({
                "system_id": block["system_id"],
                "system_name": block["system_name"],
                "system_family": block["system_family"],
            })

    return pd.Series({
        "system_id": np.nan,
        "system_name": np.nan,
        "system_family": np.nan,
    })

system_info = df_eval_final["Texto"].apply(assign_system)

df_eval_final_systems = pd.concat(
    [df_eval_final, system_info],
    axis=1
)

display(df_eval_final_systems.head())
display(df_eval_final_systems.tail())

```

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
0	1	4.0	5.0	
1	2	4.0	5.0	
2	3	5.0	5.0	

3	4	4.0	4.0
4	5	5.0	5.0

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	3.0	
4	5.0	5.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
0	5.0	5.0	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	

	Manejo de números y datos (5 3 1)	Puntaje Final	Aplico regla cero	\
0	5.0	4.75	False	
1	5.0	4.75	False	
2	5.0	5.00	False	
3	5.0	4.30	False	
4	5.0	5.00	False	

	system_id	system_name	system_family
0	prompt_1	PROMPTING_SISTEMA_1	prompting
1	prompt_1	PROMPTING_SISTEMA_1	prompting
2	prompt_1	PROMPTING_SISTEMA_1	prompting
3	prompt_1	PROMPTING_SISTEMA_1	prompting
4	prompt_1	PROMPTING_SISTEMA_1	prompting

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
139	140	4.0	4.0	
140	141	4.0	4.0	
141	142	4.0	5.0	
142	143	4.0	5.0	
143	144	0.0	0.0	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
139	5.0	5.0	
140	5.0	5.0	
141	5.0	5.0	
142	5.0	5.0	
143	0.0	0.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
139	5.0	5.0	
140	5.0	5.0	

141	5.0	5.0
142	5.0	5.0
143	0.0	0.0

	Manejo de números y datos (5 3 1)	Puntaje Final	Aplico regla cero	\
139	5.0	4.50	False	
140	5.0	4.50	False	
141	5.0	4.75	False	
142	4.0	4.70	False	
143	0.0	0.00	True	

	system_id	system_name	system_family
139	prompt_4	PROMPTING_SISTEMA_4	prompting
140	prompt_4	PROMPTING_SISTEMA_4	prompting
141	prompt_4	PROMPTING_SISTEMA_4	prompting
142	prompt_4	PROMPTING_SISTEMA_4	prompting
143	prompt_4	PROMPTING_SISTEMA_4	prompting

```
[43]: ranking_cols = [
    "system_id",
    "system_name",
    "system_family",
]

df_ranking_sistemas = (
    df_eval_final_systems
    .groupby(ranking_cols, as_index=False)
    .agg(
        n_textos=("Texto", "count"),
        puntaje_promedio=("Puntaje Final", "mean"),
        puntaje_mediano=("Puntaje Final", "median"),
        puntaje_minimo=("Puntaje Final", "min"),
        puntaje_maximo=("Puntaje Final", "max"),
        textos_con_regla_cero=("Aplico regla cero", "sum"),
    )
)

for criterion in CRITERIA_COLS:
    df_metric = (
        df_eval_final_systems
        .groupby(ranking_cols, as_index=False)[criterion]
        .mean()
        .rename(columns={criterion: f"promedio_{criterion}"})
    )

    df_ranking_sistemas = df_ranking_sistemas.merge(
        df_metric,
```

```

        on=ranking_cols,
        how="left"
    )

df_ranking_sistemas = df_ranking_sistemas.sort_values(
    "puntaje_promedio",
    ascending=False
).reset_index(drop=True)

df_ranking_sistemas["rank_cualitativo"] = np.arange(1, len(df_ranking_sistemas)
↳ + 1)

display(df_ranking_sistemas)

```

	system_id	system_name	system_family	n_textos	puntaje_promedio	\
0	prompt_1	PROMPTING_SISTEMA_1	prompting	36	4.530556	
1	prompt_2	PROMPTING_SISTEMA_2	prompting	36	4.259722	
2	prompt_3	PROMPTING_SISTEMA_3	prompting	36	2.765278	
3	prompt_4	PROMPTING_SISTEMA_4	prompting	36	2.563889	

	puntaje_mediano	puntaje_minimo	puntaje_maximo	textos_con_regla_cero	\
0	4.850	2.0	5.0	0	
1	4.675	0.0	5.0	1	
2	3.900	0.0	5.0	13	
3	4.000	0.0	5.0	15	

	promedio_Semántica (5 3 1)	promedio_Claridad (5 3 1)	\
0	4.194444	4.500000	
1	3.777778	4.111111	
2	2.444444	2.694444	
3	2.527778	2.388889	

	promedio_Fluidez y corrección (5 3 1)	promedio_Léxico y sintaxis (5 3 1)	\
0	4.694444	4.722222	
1	4.583333	4.611111	
2	2.833333	2.888889	
3	2.444444	2.777778	

	promedio_Adecuación al público / tono (5 3 1)	\
0	4.777778	
1	4.694444	
2	3.111111	
3	2.750000	

	promedio_Estructura y presentación (5 3 1)	\
0	4.694444	
1	4.583333	
2	3.000000	

3

2.805556

	promedio_Manejo de números y datos (5 3 1)	rank_cualitativo
0	4.666667	1
1	4.222222	2
2	3.111111	3
3	2.694444	4

```
[44]: eval_final_path = OUT_DIR / "evaluacion_cualitativa_prompting_conciliada.csv"
eval_final_systems_path = OUT_DIR /
↳ "evaluacion_cualitativa_prompting_conciliada_sistemas.csv"
detalle_path = OUT_DIR / "evaluacion_cualitativa_prompting_conciliada_detalle.
↳ csv"
resumen_dif_path = OUT_DIR / "resumen_diferencias_interna_externa.csv"
resumen_general_path = OUT_DIR / "resumen_general_evaluacion_conciliada.csv"
casos_dif_path = OUT_DIR / "casos_con_diferencias.csv"
casos_cero_path = OUT_DIR / "casos_regla_cero.csv"
ranking_path = OUT_DIR / "ranking_cualitativo_prompting_conciliado.csv"

df_eval_final.to_csv(eval_final_path, index=False, encoding="utf-8-sig")
df_eval_final_systems.to_csv(eval_final_systems_path, index=False,
↳ encoding="utf-8-sig")
df_conciliada.to_csv(detalle_path, index=False, encoding="utf-8-sig")
df_resumen_diferencias.to_csv(resumen_dif_path, index=False,
↳ encoding="utf-8-sig")
df_resumen_general.to_csv(resumen_general_path, index=False,
↳ encoding="utf-8-sig")
df_casos_diferencia.to_csv(casos_dif_path, index=False, encoding="utf-8-sig")
df_casos_regla_cero.to_csv(casos_cero_path, index=False, encoding="utf-8-sig")
df_ranking_sistemas.to_csv(ranking_path, index=False, encoding="utf-8-sig")

excel_path = OUT_DIR / "evaluacion_cualitativa_prompting_conciliada.xlsx"

with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
    df_resumen_general.to_excel(writer, sheet_name="Resumen general",
↳ index=False)
    df_resumen_diferencias.to_excel(writer, sheet_name="Diferencias",
↳ index=False)
    df_ranking_sistemas.to_excel(writer, sheet_name="Ranking sistemas",
↳ index=False)
    df_eval_final.to_excel(writer, sheet_name="Evaluacion final", index=False)
    df_eval_final_systems.to_excel(writer, sheet_name="Final con sistemas",
↳ index=False)
    df_conciliada.to_excel(writer, sheet_name="Detalle completo", index=False)
    df_casos_diferencia.to_excel(writer, sheet_name="Casos diferencia",
↳ index=False)
```

```
df_casos_regla_cero.to_excel(writer, sheet_name="Casos regla cero",
↪index=False)
```

```
print("Archivos guardados en:", OUT_DIR)
print("-", eval_final_path.name)
print("-", eval_final_systems_path.name)
print("-", detalle_path.name)
print("-", resumen_dif_path.name)
print("-", resumen_general_path.name)
print("-", casos_dif_path.name)
print("-", casos_cero_path.name)
print("-", ranking_path.name)
print("-", excel_path.name)
```

Archivos guardados en: /home/harielpadillasanchez/Documentos/TT/TT2/outputs/evaluacion_cualitativa_conciliada

- evaluacion_cualitativa_prompting_conciliada.csv
- evaluacion_cualitativa_prompting_conciliada_sistemas.csv
- evaluacion_cualitativa_prompting_conciliada_detalle.csv
- resumen_diferencias_interna_externa.csv
- resumen_general_evaluacion_conciliada.csv
- casos_con_diferencias.csv
- casos_regla_cero.csv
- ranking_cualitativo_prompting_conciliado.csv
- evaluacion_cualitativa_prompting_conciliada.xlsx

```
[45]: import pandas as pd
from pathlib import Path

def explorar_hojas_excel(path):
    path = Path(path)

    print("=" * 100)
    print("Archivo:", path.name)
    print("Ruta:", path)
    print("Existe:", path.exists())

    if not path.exists():
        return

    xls = pd.ExcelFile(path)
    print("Hojas encontradas:")
    for i, sheet in enumerate(xls.sheet_names, start=1):
        print(f"{i}. {sheet}")

explorar_hojas_excel(FILE_INTERNA)
```

```
explorar_hojas_excel(FILE_EXTERNA)
```

```
=====
=====
Archivo: Evaluacion_cualitativa_interna.xlsx
Ruta: /home/harielpadillasanchez/Documentos/TT/TT2/data/cualitativa/Evaluacion_c
ualitativa_interna.xlsx
Existe: True
Hojas encontradas:
1. Rúbrica
2. Textos simplificados
3. Evaluación cualitativa
4. Textos simplificados Fine-tunin
5. Evaluación cualitativa fine tun
=====
=====
Archivo: Evaluacion_cualitativa_externa.xlsx
Ruta: /home/harielpadillasanchez/Documentos/TT/TT2/data/cualitativa/Evaluacion_c
ualitativa_externa.xlsx
Existe: True
Hojas encontradas:
1. Rúbrica
2. Textos simplificados
3. Evaluación cualitativa
4. Textos simplificados Fine-tunin
5. Evaluación cualitativa fine tun
```

```
[46]: def explorar_estructura_excel(path):
    path = Path(path)

    print("=" * 100)
    print("Archivo:", path.name)

    xls = pd.ExcelFile(path)

    for sheet in xls.sheet_names:
        print("\n" + "-" * 80)
        print("Hoja:", sheet)

        df = pd.read_excel(path, sheet_name=sheet)

        print("Shape:", df.shape)
        print("Columnas:")
        for col in df.columns:
            print(f"- {repr(col)}")

    explorar_estructura_excel(FILE_INTERNA)
```

```
explorar_estructura_excel(FILE_EXTERNA)
```

```
=====
```

```
Archivo: Evaluacion_cualitativa_interna.xlsx
```

```
-----
```

Hoja: Rúbrica

Shape: (8, 6)

Columnas:

- 'Criterio'
- 'Peso'
- 'Qué se evalúa'
- 'Nivel 5'
- 'Nivel 3'
- 'Nivel 1'

```
-----
```

Hoja: Textos simplificados

Shape: (144, 6)

Columnas:

- 'id'
- 'Texto original'
- 'Texto de referencia'
- 'Texto simplificado por el LLM'
- 'Columna 1'
- 'Columna 2'

```
-----
```

Hoja: Evaluación cualitativa

Shape: (145, 11)

Columnas:

- 'Texto'
- 'Semántica (5|3|1)'
- 'Claridad (5|3|1)'
- 'Fluidez y corrección (5|3|1)'
- 'Léxico y sintaxis (5|3|1)'
- 'Adecuación al público / tono (5|3|1)'
- 'Estructura y presentación (5|3|1)'
- 'Manejo de números y datos (5|3|1)'
- 'Puntaje Final'
- 'Unnamed: 9'
- 'Unnamed: 10'

```
-----
```

Hoja: Textos simplificados Fine-tunin

Shape: (72, 4)

Columnas:

- 'id'
- 'Texto original'
- 'Texto de referencia'
- 'Texto simplificado por el LLM'

Hoja: Evaluación cualitativa fine tun

Shape: (74, 10)

Columnas:

- 'Texto'
- 'Semántica (5|3|1)'
- 'Claridad (5|3|1)'
- 'Fluidez y corrección (5|3|1)'
- 'Léxico y sintaxis (5|3|1)'
- 'Adecuación al público / tono (5|3|1)'
- 'Estructura y presentación (5|3|1)'
- 'Manejo de números y datos (5|3|1)'
- 'Puntaje Final'
- 'Unnamed: 9'

=====

=====

Archivo: Evaluacion_cualitativa_externa.xlsx

Hoja: Rúbrica

Shape: (8, 6)

Columnas:

- 'Criterio'
- 'Peso'
- 'Qué se evalúa'
- 'Nivel 5'
- 'Nivel 3'
- 'Nivel 1'

Hoja: Textos simplificados

Shape: (144, 6)

Columnas:

- 'id'
- 'Texto original'
- 'Texto de referencia'
- 'Texto simplificado por el LLM'
- 'Columna 1'
- 'Columna 2'

Hoja: Evaluación cualitativa

Shape: (145, 11)

Columnas:

- 'Texto'
- 'Semántica (5|3|1)'
- 'Claridad (5|3|1)'
- 'Fluidez y corrección (5|3|1)'
- 'Léxico y sintaxis (5|3|1)'
- 'Adecuación al público / tono (5|3|1)'
- 'Estructura y presentación (5|3|1)'
- 'Manejo de números y datos (5|3|1)'
- 'Puntaje Final'
- 'Unnamed: 9'
- 'Unnamed: 10'

Hoja: Textos simplificados Fine-tunin

Shape: (72, 4)

Columnas:

- 'id'
- 'Texto original'
- 'Texto de referencia'
- 'Texto simplificado por el LLM'

Hoja: Evaluación cualitativa fine tun

Shape: (644, 11)

Columnas:

- 'Texto'
- 'Semántica (5|3|1)'
- 'Claridad (5|3|1)'
- 'Fluidez y corrección (5|3|1)'
- 'Léxico y sintaxis (5|3|1)'
- 'Adecuación al público / tono (5|3|1)'
- 'Estructura y presentación (5|3|1)'
- 'Manejo de números y datos (5|3|1)'
- 'Puntaje Final'
- 'Unnamed: 9'
- 'Unnamed: 10'

[47]: SHEET_NAME = "Evaluación cualitativa"

```
df_raw_interna = pd.read_excel(FILE INTERNA, sheet_name=SHEET_NAME)
df_raw_externa = pd.read_excel(FILE EXTERNA, sheet_name=SHEET_NAME)

print("INterna raw shape:", df_raw_interna.shape)
display(df_raw_interna.head(15))

print("Externa raw shape:", df_raw_externa.shape)
```

```
display(df_raw_externa.head(15))
```

INterna raw shape: (145, 11)

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
0	Peso	0.25	0.25	
1	1	3.00	5.00	
2	2	3.00	5.00	
3	3	5.00	5.00	
4	4	3.00	5.00	
5	5	5.00	5.00	
6	6	5.00	5.00	
7	7	5.00	5.00	
8	8	5.00	5.00	
9	9	5.00	5.00	
10	10	3.00	5.00	
11	11	3.00	5.00	
12	12	3.00	5.00	
13	13	5.00	5.00	
14	14	5.00	5.00	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
0	0.15	0.1	
1	5.00	5.0	
2	5.00	5.0	
3	5.00	5.0	
4	5.00	5.0	
5	5.00	5.0	
6	5.00	5.0	
7	5.00	5.0	
8	5.00	5.0	
9	5.00	5.0	
10	5.00	5.0	
11	5.00	5.0	
12	5.00	5.0	
13	5.00	5.0	
14	5.00	5.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
0	0.1	0.1	
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	
5	5.0	5.0	
6	5.0	5.0	
7	5.0	5.0	
8	5.0	5.0	

9	5.0	5.0
10	5.0	5.0
11	5.0	5.0
12	5.0	5.0
13	5.0	5.0
14	5.0	5.0

	Manejo de números y datos (5 3 1)	Puntaje Final	Unnamed: 9	Unnamed: 10
0	0.05	1.0	NaN	NaN
1	5.00	4.5	NaN	NaN
2	5.00	4.5	NaN	NaN
3	5.00	5.0	NaN	NaN
4	5.00	4.5	NaN	NaN
5	5.00	5.0	NaN	NaN
6	5.00	5.0	NaN	NaN
7	5.00	5.0	NaN	NaN
8	5.00	5.0	NaN	NaN
9	5.00	5.0	NaN	NaN
10	5.00	4.5	NaN	NaN
11	5.00	4.5	NaN	NaN
12	5.00	4.5	NaN	NaN
13	5.00	5.0	NaN	NaN
14	5.00	5.0	NaN	NaN

Externa raw shape: (145, 11)

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
0	Peso	0.25	0.25	
1	1	5.00	5.00	
2	2	5.00	5.00	
3	3	5.00	5.00	
4	4	5.00	3.00	
5	5	5.00	5.00	
6	6	5.00	5.00	
7	7	5.00	5.00	
8	8	5.00	3.00	
9	9	5.00	5.00	
10	10	5.00	5.00	
11	11	5.00	5.00	
12	12	5.00	5.00	
13	13	5.00	5.00	
14	14	5.00	5.00	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
0	0.15	0.1	
1	5.00	5.0	
2	5.00	5.0	
3	5.00	5.0	
4	5.00	1.0	

5	5.00	5.0
6	5.00	5.0
7	5.00	5.0
8	5.00	5.0
9	5.00	5.0
10	5.00	5.0
11	5.00	5.0
12	5.00	5.0
13	5.00	5.0
14	3.00	5.0

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
0	0.1		0.1
1	5.0		5.0
2	5.0		5.0
3	5.0		5.0
4	5.0		5.0
5	5.0		5.0
6	5.0		5.0
7	5.0		5.0
8	5.0		3.0
9	5.0		5.0
10	5.0		5.0
11	5.0		5.0
12	5.0		5.0
13	5.0		5.0
14	5.0		5.0

	Manejo de números y datos (5 3 1)	Puntaje Final	Unnamed: 9	Unnamed: 10
0	0.05	1.0	NaN	NaN
1	5.00	5.0	NaN	NaN
2	5.00	5.0	NaN	NaN
3	5.00	5.0	NaN	NaN
4	5.00	4.1	NaN	NaN
5	5.00	5.0	NaN	NaN
6	5.00	5.0	NaN	NaN
7	5.00	5.0	NaN	NaN
8	5.00	4.3	NaN	NaN
9	5.00	5.0	NaN	NaN
10	5.00	5.0	NaN	NaN
11	5.00	5.0	NaN	NaN
12	5.00	5.0	NaN	NaN
13	5.00	5.0	NaN	NaN
14	5.00	4.7	NaN	NaN

```
[48]: print("Columnas internas:")
      for i, col in enumerate(df_raw_interna.columns):
```

```

    print(i, repr(col))

print("\nColumnas externas:")
for i, col in enumerate(df_raw_externa.columns):
    print(i, repr(col))

```

Columnas internas:

```

0 'Texto'
1 'Semántica (5|3|1)'
2 'Claridad (5|3|1)'
3 'Fluidez y corrección (5|3|1)'
4 'Léxico y sintaxis (5|3|1)'
5 'Adecuación al público / tono (5|3|1)'
6 'Estructura y presentación (5|3|1)'
7 'Manejo de números y datos (5|3|1)'
8 'Puntaje Final'
9 'Unnamed: 9'
10 'Unnamed: 10'

```

Columnas externas:

```

0 'Texto'
1 'Semántica (5|3|1)'
2 'Claridad (5|3|1)'
3 'Fluidez y corrección (5|3|1)'
4 'Léxico y sintaxis (5|3|1)'
5 'Adecuación al público / tono (5|3|1)'
6 'Estructura y presentación (5|3|1)'
7 'Manejo de números y datos (5|3|1)'
8 'Puntaje Final'
9 'Unnamed: 9'
10 'Unnamed: 10'

```

[49]: TEXT_COL = "Texto"

```

print("¿Existe columna Texto en interna?", TEXT_COL in df_raw_interna.columns)
print("¿Existe columna Texto en externa?", TEXT_COL in df_raw_externa.columns)

if TEXT_COL in df_raw_interna.columns:
    print("\nValores únicos iniciales de Texto en interna:")
    display(df_raw_interna[TEXT_COL].head(20))
    print("No nulos:", df_raw_interna[TEXT_COL].notna().sum())
    print("Numéricos convertibles:", pd.to_numeric(df_raw_interna[TEXT_COL],
    ↪errors="coerce").notna().sum())

if TEXT_COL in df_raw_externa.columns:
    print("\nValores únicos iniciales de Texto en externa:")
    display(df_raw_externa[TEXT_COL].head(20))

```

```
print("No nulos:", df_raw_externa[TEXT_COL].notna().sum())
print("Numéricos convertibles:", pd.to_numeric(df_raw_externa[TEXT_COL],
↪errors="coerce").notna().sum())
```

¿Existe columna Texto en interna? True

¿Existe columna Texto en externa? True

Valores únicos iniciales de Texto en interna:

	Peso
0	
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19

Name: Texto, dtype: object

No nulos: 145

Numéricos convertibles: 144

Valores únicos iniciales de Texto en externa:

	Peso
0	
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12

```

13      13
14      14
15      15
16      16
17      17
18      18
19      19
Name: Texto, dtype: object

No nulos: 145
Numéricos convertibles: 144

```

```

[50]: CRITERIA_COLS = [
    "Semántica (5|3|1)",
    "Claridad (5|3|1)",
    "Fluidez y corrección (5|3|1)",
    "Léxico y sintaxis (5|3|1)",
    "Adecuación al público / tono (5|3|1)",
    "Estructura y presentación (5|3|1)",
    "Manejo de números y datos (5|3|1)",
]

print("Revisión de criterios en interna:")
for col in CRITERIA_COLS:
    print("\n", repr(col))
    if col in df_raw_interna.columns:
        print("No nulos:", df_raw_interna[col].notna().sum())
        print("Valores únicos:", sorted(pd.to_numeric(df_raw_interna[col],
errors="coerce").dropna().unique().tolist()))
    else:
        print("No existe")

print("\n" + "=" * 100)
print("Revisión de criterios en externa:")
for col in CRITERIA_COLS:
    print("\n", repr(col))
    if col in df_raw_externa.columns:
        print("No nulos:", df_raw_externa[col].notna().sum())
        print("Valores únicos:", sorted(pd.to_numeric(df_raw_externa[col],
errors="coerce").dropna().unique().tolist()))
    else:
        print("No existe")

```

Revisión de criterios en interna:

```

'Semántica (5|3|1)'
No existe

```

'Claridad (5|3|1)'

No existe

'Fluidez y corrección (5|3|1)'

No existe

'Léxico y sintaxis (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.1, 3.0, 5.0]

'Adecuación al público / tono (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.1, 5.0]

'Estructura y presentación (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.1, 3.0, 5.0]

'Manejo de números y datos (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.05, 1.0, 3.0, 5.0]

=====

Revisión de criterios en externa:

'Semántica (5|3|1)'

No existe

'Claridad (5|3|1)'

No existe

'Fluidez y corrección (5|3|1)'

No existe

'Léxico y sintaxis (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.1, 1.0, 3.0, 5.0]

'Adecuación al público / tono (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.1, 1.0, 3.0, 5.0]

'Estructura y presentación (5|3|1)'

No nulos: 145

Valores únicos: [0.0, 0.1, 1.0, 3.0, 5.0]

'Manejo de números y datos (5|3|1)'

No nulos: 143

Valores únicos: [0.0, 0.05, 1.0, 3.0, 5.0]

```
[51]: VALID_SCORES = {0, 1, 3, 5}

def detectar_filas_evaluacion(df, criteria_cols):
    temp = df.copy()

    for col in criteria_cols:
        if col in temp.columns:
            temp[col] = pd.to_numeric(temp[col], errors="coerce")

    existing_criteria = [col for col in criteria_cols if col in temp.columns]

    def tiene_valor_valido(row):
        values = row[existing_criteria].dropna().tolist()
        return any(v in VALID_SCORES for v in values)

    mask = temp.apply(tiene_valor_valido, axis=1)

    return temp[mask].copy()

df_detectada_interna = detectar_filas_evaluacion(df_raw_interna, CRITERIA_COLS)
df_detectada_externa = detectar_filas_evaluacion(df_raw_externa, CRITERIA_COLS)

print("Filas detectadas como evaluación real en interna:", df_detectada_interna.
      ↪shape)
display(df_detectada_interna.head(10))

print("Filas detectadas como evaluación real en externa:", df_detectada_externa.
      ↪shape)
display(df_detectada_externa.head(10))
```

Filas detectadas como evaluación real en interna: (144, 11)

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
1	1	3.0	5.0	
2	2	3.0	5.0	
3	3	5.0	5.0	
4	4	3.0	5.0	
5	5	5.0	5.0	
6	6	5.0	5.0	
7	7	5.0	5.0	
8	8	5.0	5.0	
9	9	5.0	5.0	
10	10	3.0	5.0	

Fluidez y corrección (5|3|1) Léxico y sintaxis (5|3|1) \

1	5.0	5.0
2	5.0	5.0
3	5.0	5.0
4	5.0	5.0
5	5.0	5.0
6	5.0	5.0
7	5.0	5.0
8	5.0	5.0
9	5.0	5.0
10	5.0	5.0

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	
5	5.0	5.0	
6	5.0	5.0	
7	5.0	5.0	
8	5.0	5.0	
9	5.0	5.0	
10	5.0	5.0	

	Manejo de números y datos (5 3 1)	Puntaje Final	Unnamed: 9	Unnamed: 10
1	5.0	4.5	NaN	NaN
2	5.0	4.5	NaN	NaN
3	5.0	5.0	NaN	NaN
4	5.0	4.5	NaN	NaN
5	5.0	5.0	NaN	NaN
6	5.0	5.0	NaN	NaN
7	5.0	5.0	NaN	NaN
8	5.0	5.0	NaN	NaN
9	5.0	5.0	NaN	NaN
10	5.0	4.5	NaN	NaN

Filas detectadas como evaluación real en externa: (144, 11)

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
1	1	5.0	5.0	
2	2	5.0	5.0	
3	3	5.0	5.0	
4	4	5.0	3.0	
5	5	5.0	5.0	
6	6	5.0	5.0	
7	7	5.0	5.0	
8	8	5.0	3.0	
9	9	5.0	5.0	
10	10	5.0	5.0	

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	1.0	
5	5.0	5.0	
6	5.0	5.0	
7	5.0	5.0	
8	5.0	5.0	
9	5.0	5.0	
10	5.0	5.0	

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
1	5.0	5.0	
2	5.0	5.0	
3	5.0	5.0	
4	5.0	5.0	
5	5.0	5.0	
6	5.0	5.0	
7	5.0	5.0	
8	5.0	3.0	
9	5.0	5.0	
10	5.0	5.0	

	Manejo de números y datos (5 3 1)	Puntaje Final	Unnamed: 9	Unnamed: 10
1	5.0	5.0	NaN	NaN
2	5.0	5.0	NaN	NaN
3	5.0	5.0	NaN	NaN
4	5.0	4.1	NaN	NaN
5	5.0	5.0	NaN	NaN
6	5.0	5.0	NaN	NaN
7	5.0	5.0	NaN	NaN
8	5.0	4.3	NaN	NaN
9	5.0	5.0	NaN	NaN
10	5.0	5.0	NaN	NaN

```
[52]: print("Interna detectada:")
print("Número de filas:", len(df_detectada_interna))

if len(df_detectada_interna) >= 144:
    df_preview_interna = df_detectada_interna.copy().reset_index(drop=True)
    df_preview_interna["Texto_asignado_por_orden"] = range(1,
↪len(df_preview_interna) + 1)

    display(df_preview_interna.head(5))
    display(df_preview_interna.tail(5))
else:
```

```
print("No se detectaron 144 filas. Hay que revisar manualmente la
estructura.")
```

Interna detectada:

Número de filas: 144

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	Fluidez y corrección (5 3 1)	\
0	1	3.0		5.0	
1	2	3.0		5.0	
2	3	5.0		5.0	
3	4	3.0		5.0	
4	5	5.0		5.0	

	Léxico y sintaxis (5 3 1)	Adecuación al público / tono (5 3 1)	\
0	5.0		5.0
1	5.0		5.0
2	5.0		5.0
3	5.0		5.0
4	5.0		5.0

	Estructura y presentación (5 3 1)	Manejo de números y datos (5 3 1)	\
0		5.0	5.0
1		5.0	5.0
2		5.0	5.0
3		5.0	5.0
4		5.0	5.0

	Puntaje Final	Unnamed: 9	Unnamed: 10	Texto_asignado_por_orden
0	4.5	NaN	NaN	1
1	4.5	NaN	NaN	2
2	5.0	NaN	NaN	3
3	4.5	NaN	NaN	4
4	5.0	NaN	NaN	5

	Texto	Semántica (5 3 1)	Claridad (5 3 1)	\
139	140	3.0		3.0
140	141	3.0		3.0
141	142	5.0		5.0
142	143	5.0		5.0
143	144	3.0		3.0

	Fluidez y corrección (5 3 1)	Léxico y sintaxis (5 3 1)	\
139		5.0	5.0
140		5.0	5.0
141		5.0	5.0
142		5.0	5.0
143		5.0	5.0

	Adecuación al público / tono (5 3 1)	Estructura y presentación (5 3 1)	\
--	--------------------------------------	-----------------------------------	---

139	5.0	5.0
140	5.0	5.0
141	5.0	5.0
142	5.0	5.0
143	5.0	5.0

	Manejo de números y datos (5 3 1)	Puntaje Final	\
139	5.0	4.0	
140	5.0	4.0	
141	5.0	5.0	
142	3.0	4.9	
143	5.0	4.0	

		Unnamed: 9	Unnamed: 10	\
139	cambia u omite palabras que cortan un poco el ...		NaN	
140	si mantiene la idea pero mete cosas que no est...		NaN	
141	bien solo mete un signo extra		NaN	
142	le falto simplificar un numero y metio un cara...		NaN	
143	metio traduccion en ingles y caracter extra		NaN	

	Texto_asignado_por_orden
139	140
140	141
141	142
142	143
143	144